ECE 491 Image Processing and Computer Vision (Fall 2026)



Lecture 24 - Object Detection

Depart. of ECE, NC State University

Instructor: Tianfu (Matt) Wu

Outline

- **Topics:** from classification to detection; datasets and metrics (IoU, NMS, mAP); sliding windows and region proposals; R-CNN, Fast R-CNN, Faster R-CNN; one-stage detectors (YOLO, SSD, RetinaNet); DETR.
- **Key concept:** a detector outputs a variable-length set of (box, class, score). Dense candidates plus non-maximum suppression turn a classifier into a detector.
- **Notes:** notebook 28 (marimo): IoU and NMS by hand, four pretrained torchvision detectors, a YOLO-style detector trained from scratch, and mAP.

So far, Image Classification

Classification



CAT

From Image Level to Object Level

Classification

CAT

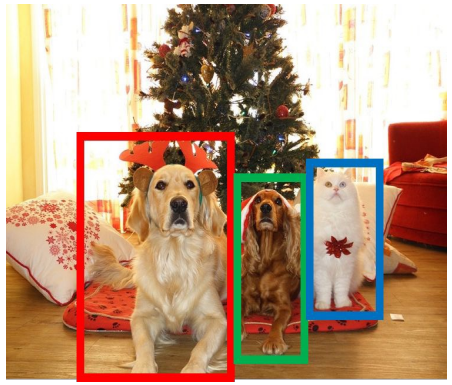
No spatial extent

Semantic Segmentation

GRASS, CAT, TREE, SKY

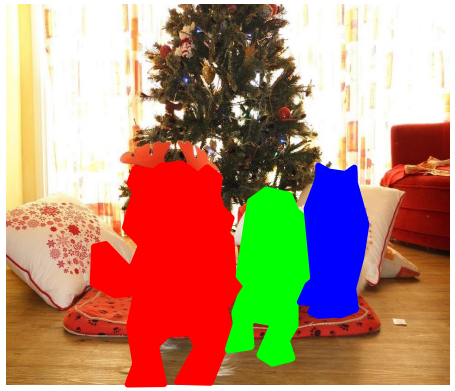
No objects, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

Multiple Object

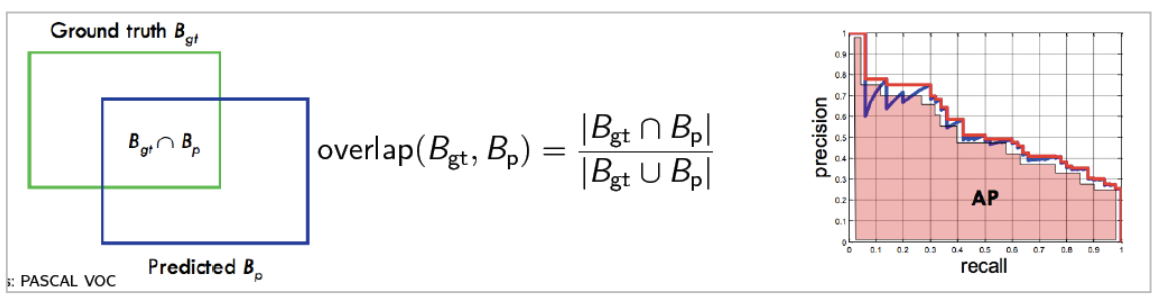
This image is CC0 public domain

Data and Metrics

- E.g., PASCAL VOC, 20 Categories

What are where?

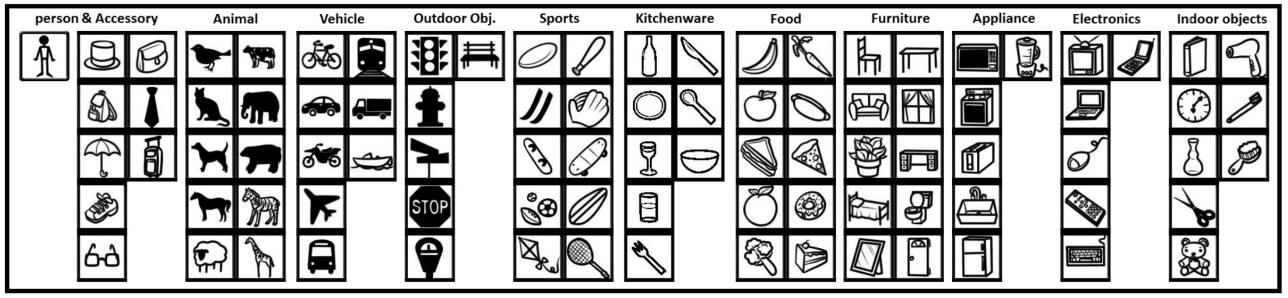


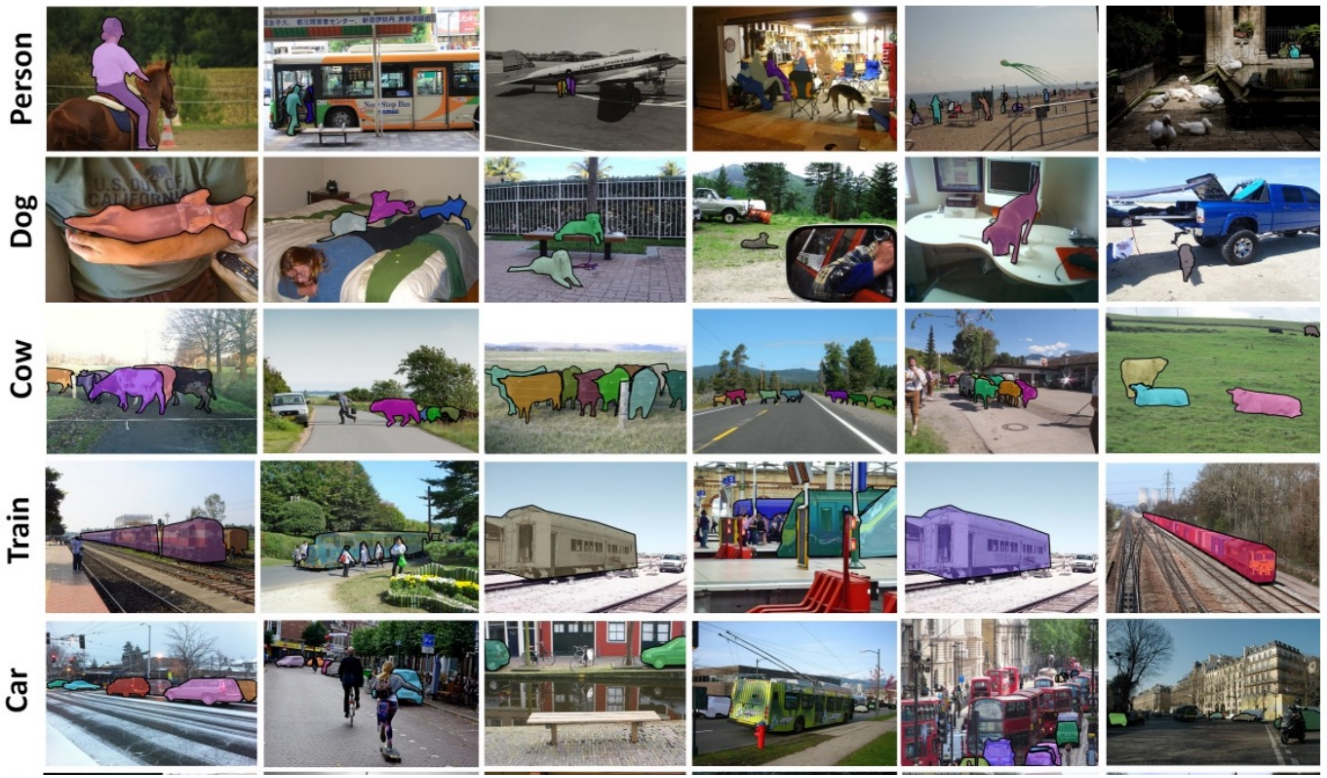


- Everingham, M., Van Gool, L., Williams, C. K. I., Winn, J. and Zisserman, A. *The PASCAL Visual Object Classes (VOC) Challenge*. IJCV, 2010

Data and Metrics

- E.g., MS-COCO, 11 super categories and 91 categories





Average Precision (AP):

AP	% AP at IoU=.50:.05:.95 **(primary challenge metric)**
$AP^{IoU=.50}$	% AP at IoU=.50 (PASCAL VOC metric)
$AP^{IoU=.75}$	% AP at IoU=.75 (strict metric)

AP Across Scales:

AP^{small}	% AP for small objects: area < 32^2
AP^{medium}	% AP for medium objects: 32^2 < area < 96^2
AP^{large}	% AP for large objects: area > 96^2

Average Recall (AR):

$AR^{max=1}$	% AR given 1 detection per image
$AR^{max=10}$	% AR given 10 detections per image
$AR^{max=100}$	% AR given 100 detections per image

AR Across Scales:

AR^{small}	% AR for small objects: area < 32^2
AR^{medium}	% AR for medium objects: 32^2 < area < 96^2
AR^{large}	% AR for large objects: area > 96^2

- Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, Piotr Dollár, *Microsoft COCO: Common Objects in Context*, arXiv 1405.0312, 2014

https://paperswithcode.com/datasets

IoU and Non-Maximum Suppression

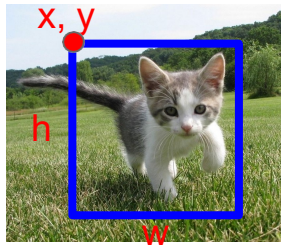
- **IoU** = area(prediction ∩ ground truth) / area(prediction ∪ ground truth). A detection counts as correct if IoU ≥ 0.5 (PASCAL VOC).
  - Two equal boxes shifted sideways by s: IoU = (w − s) / (w + s). IoU ≥ 0.5 tolerates a shift of w/3; IoU ≥ 0.75 only w/7.
- **NMS:** sort the boxes by score; keep the best; delete every box of the same class that overlaps it with IoU above a threshold (typically 0.5–0.7); repeat.
- **Notebook 28:** Faster R-CNN on a COCO living-room photo outputs 198 boxes with score ≥ 0.5; NMS at IoU 0.5 keeps 18.

Evaluating Detectors: AP and mAP

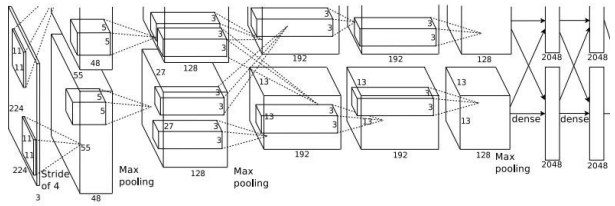
- **Matching:** sort a class's detections by score. Each is a true positive if it overlaps a not-yet-matched ground-truth box with IoU $\geq$ threshold, otherwise a false positive (duplicates included).
- **AP and mAP:** AP is the area under the precision–recall curve traced as the score threshold falls; mAP is the mean AP over classes.
- **PASCAL VOC vs. COCO:** VOC reports mAP at IoU 0.5; COCO averages mAP over IoU 0.50, 0.55, …, 0.95, which rewards tight boxes.
- **Notebook 28 (toy YOLO):** mAP@0.5 = 0.969, mAP@0.75 = 0.823, COCO-style 0.660: finding objects is easier than boxing them tightly.

Object Detection

- Consider single object: classification + localization



This image is CC0 public domain



Vector: 4096

Fully Connected: 4096 to 1000

Class Scores
Cat: 0.9
Dog: 0.05
Car: 0.01
...

Softmax Loss

Correct label: Cat

Fully Connected: 4096 to 4

Box Coordinates (x, y, w, h)

L2 Loss

Correct box: (x', y', w', h')

Multitask Loss

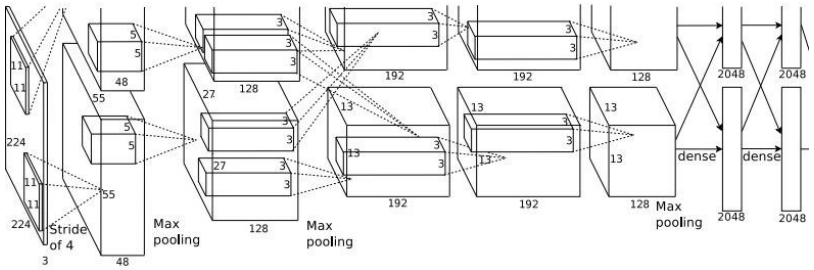
+ → **Loss**

Treat localization as a regression problem!

Object Detection

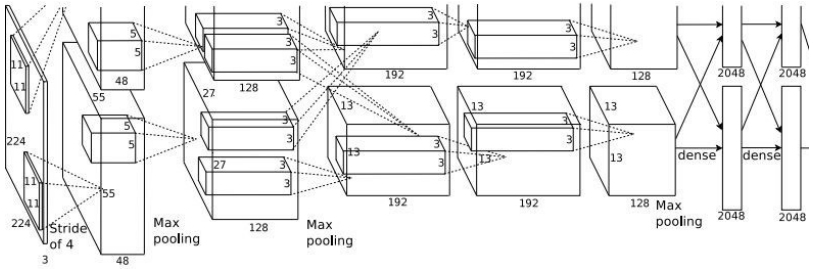
- Multiple Objects





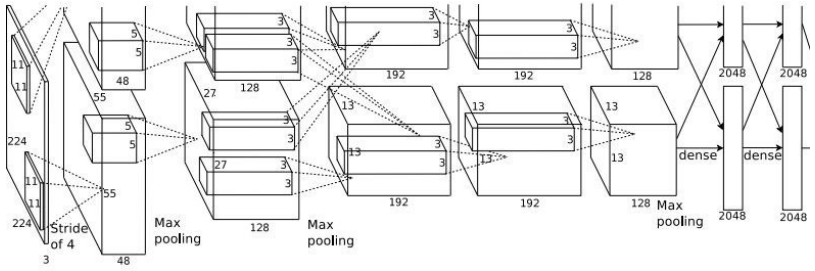
CAT: (x, y, w, h)





DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)





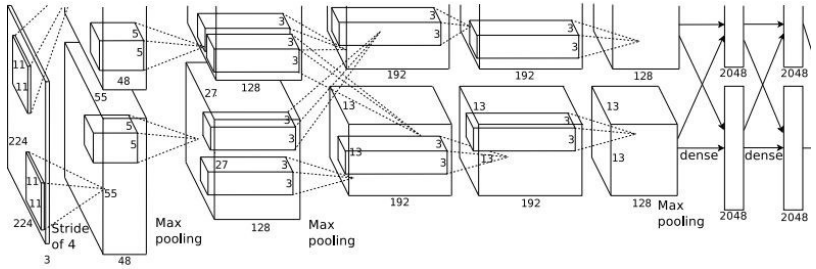
DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

Object Detection

- Multiple Objects

Each image needs a different number of outputs!

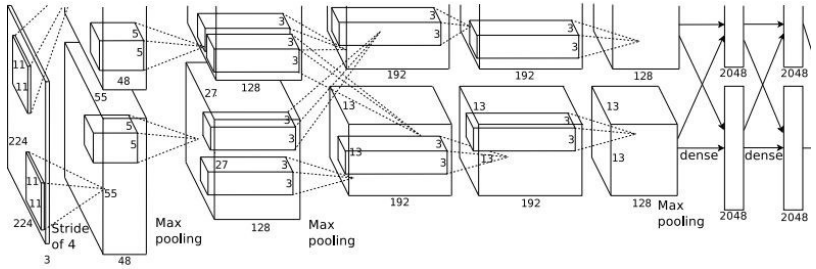




CAT: (x, y, w, h)

4 numbers

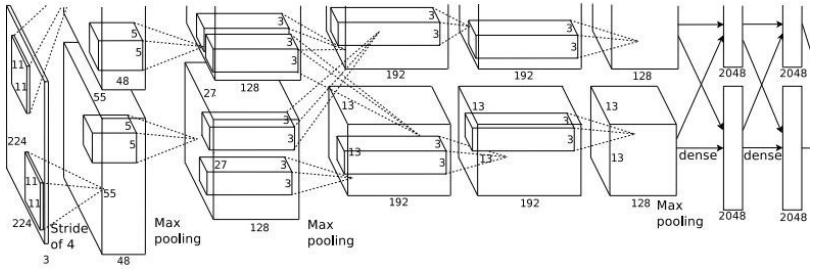




DOG: (x, y, w, h)
DOG: (x, y, w, h)
CAT: (x, y, w, h)

12 numbers





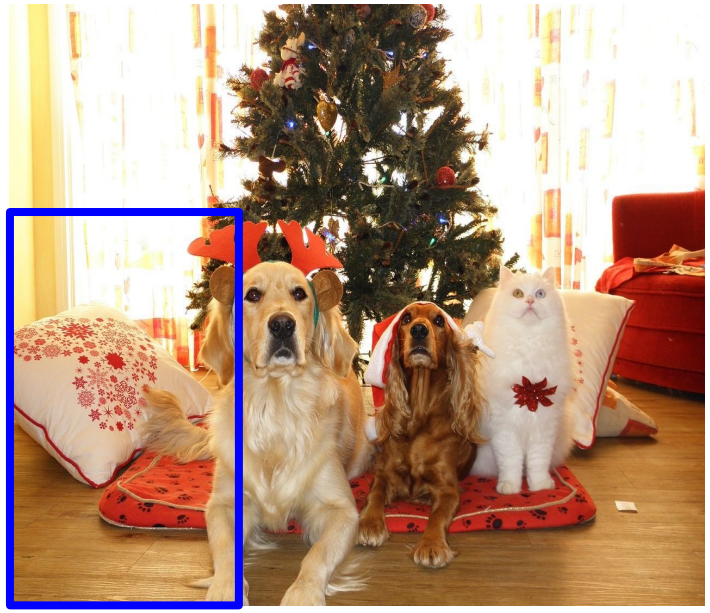
DUCK: (x, y, w, h)
DUCK: (x, y, w, h)
….

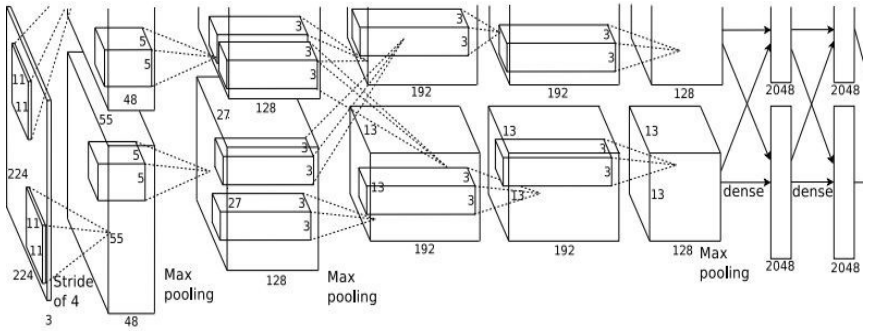
Many numbers!

Object Detection

- Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background



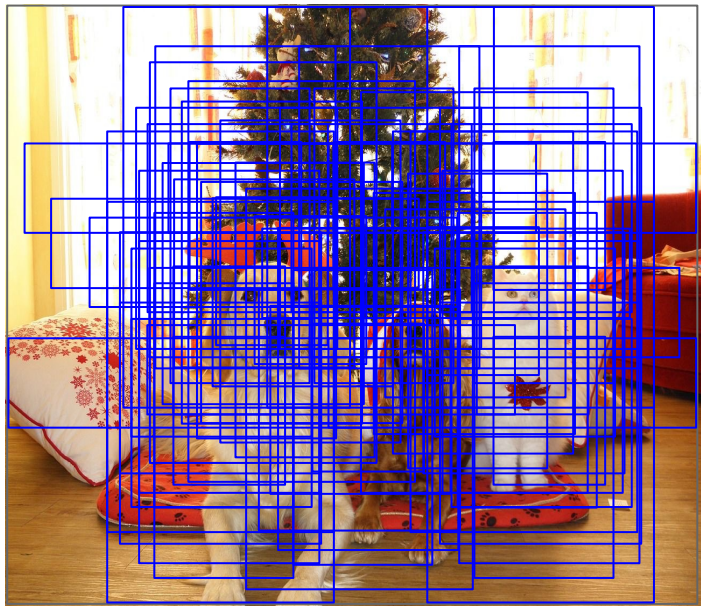


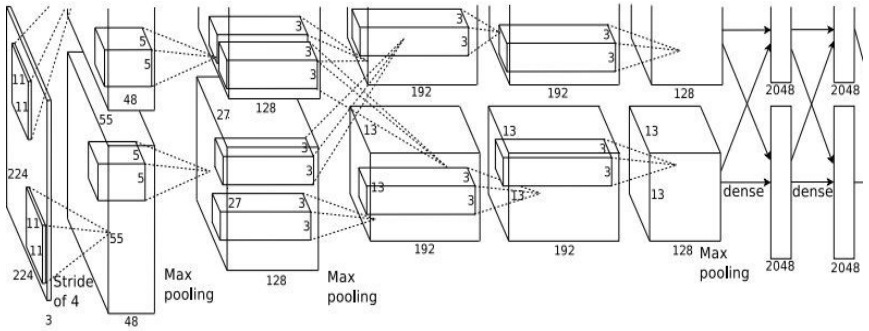
Dog? NO
Cat? NO
Background? YES

Object Detection

- Multiple Objects

Apply a CNN to many different crops of the image, CNN classifies each crop as object or background





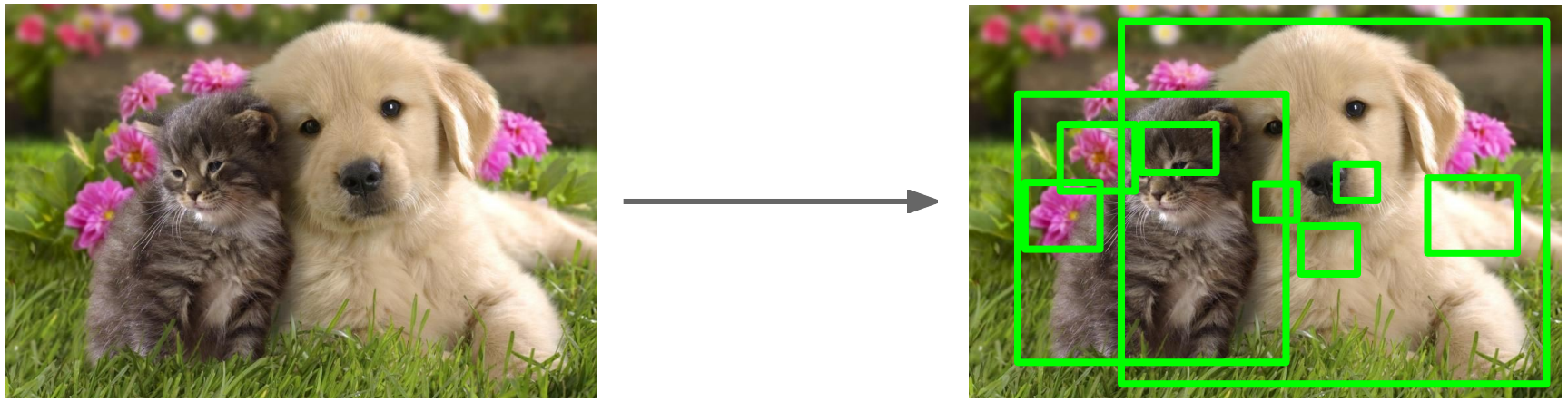
Dog? NO
Cat? YES
Background? NO

Problem: Need to apply CNN to huge number of locations, scales, and aspect ratios, very computationally expensive!

Object Detection

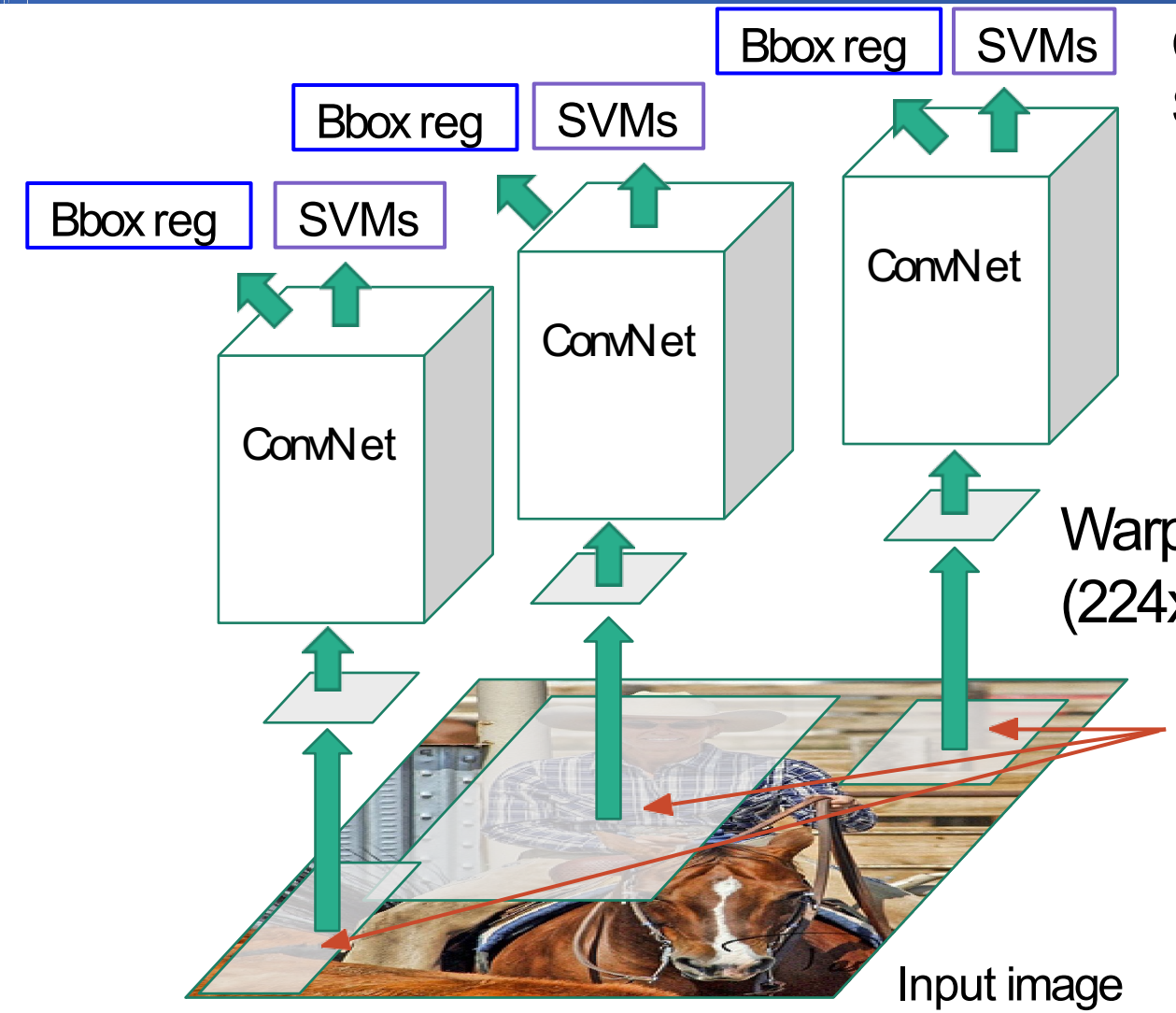
- Multiple Objects
  - Region proposals via selective search

- Find “blobby” image regions that are likely to contain objects
- Relatively fast to run; e.g. Selective Search gives 2000 region proposals in a few seconds on CPU



Alexe et al, “Measuring the objectness of image windows”, TPAMI 2012
Uijlings et al, “Selective Search for Object Recognition”, IJCV 2013
Cheng et al, “BING: Binarized normed gradients for objectness estimation at 300fps”, CVPR 2014
Zitnick and Dollar, “Edge boxes: Locating object proposals from edges”, ECCV 2014

R-CNN



Classify regions with SVMs

Forward each region through ConvNet (ImageNet-pretranied)

Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

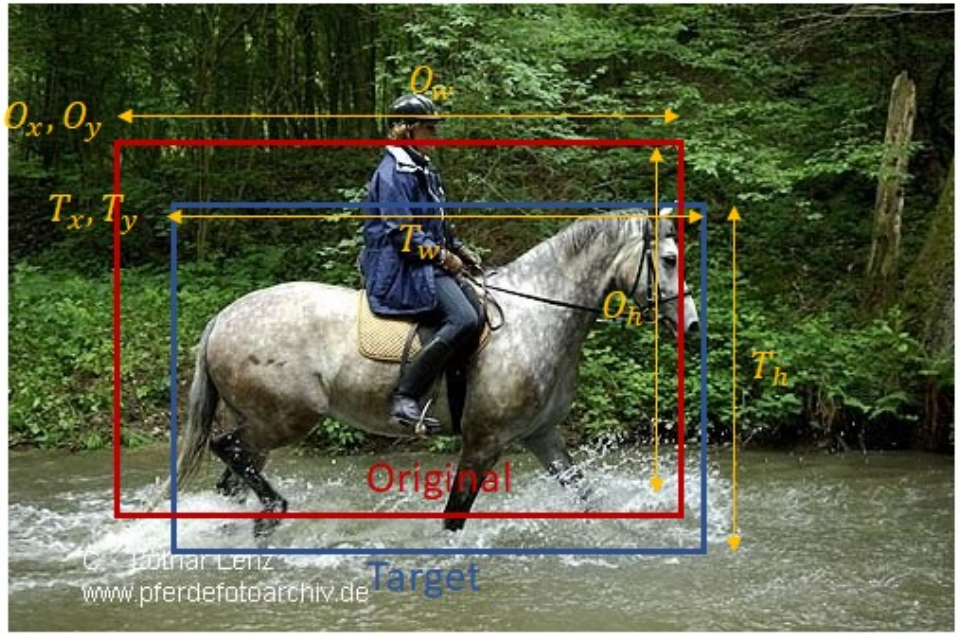
Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Bbox reg: Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)

R-CNN

- Bbox Regression

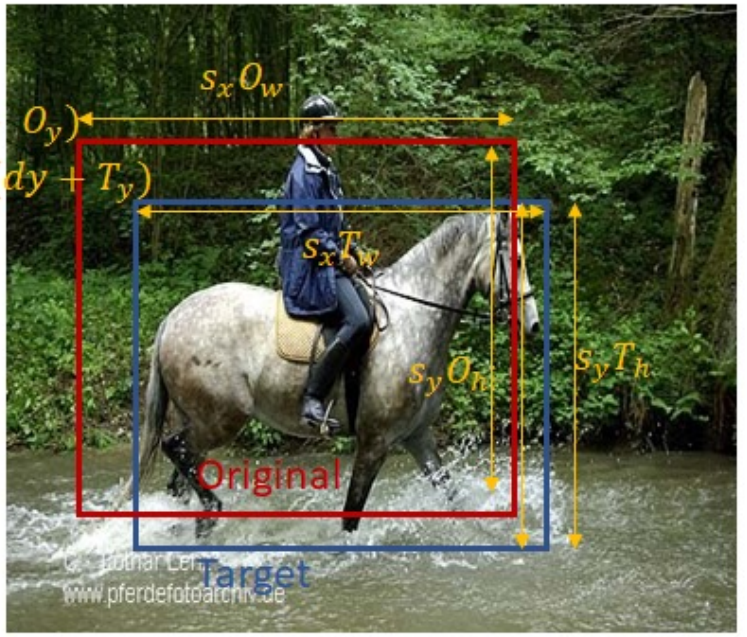
Target: Ground Truth Bounding Box
Original: RPN Bounding Box



$s_x(dx + O_x), s_y(dy + O_y)$
$s_x(dx + T_x), s_y(dy + T_y)$

Apply affine transformation:

$$\begin{bmatrix} s_x & 0 & dx \\ 0 & s_y & dy \end{bmatrix}$$



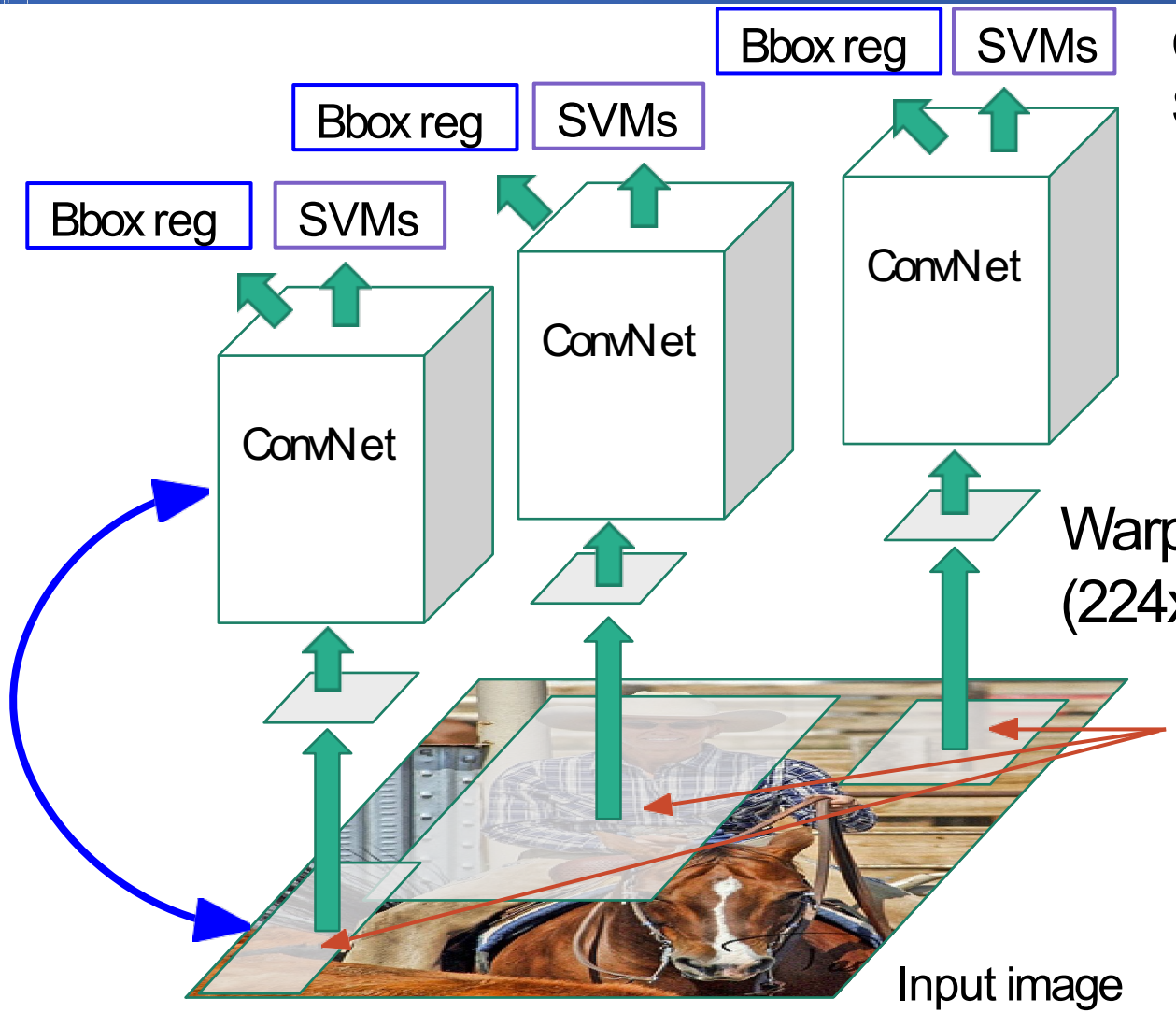
Regression Coefficients:

$$t_x = \frac{T_x - O_x}{O_w}, t_y = \frac{T_y - O_y}{O_h},$$
$$t_w = \log\left(\frac{T_w}{O_w}\right), t_h = \log(\frac{T_h}{O_h})$$

Regression Coefficients stay the same:

$$t_x = \frac{s_x(T_x + dx - (O_x + dx))}{s_x O_h}, t_y = \frac{s_y(T_y + dy - (O_y + dy))}{s_y O_y},$$
$$t_w = \log\left(\frac{s_x T_w}{s_x O_w}\right), t_h = \log(\frac{s_y T_h}{s_y O_h})$$

"Slow" R-CNN



Classify regions with SVMs

Forward each region through ConvNet

Warped image regions (224x224 pixels)

Regions of Interest (RoI) from a proposal method (~2k)

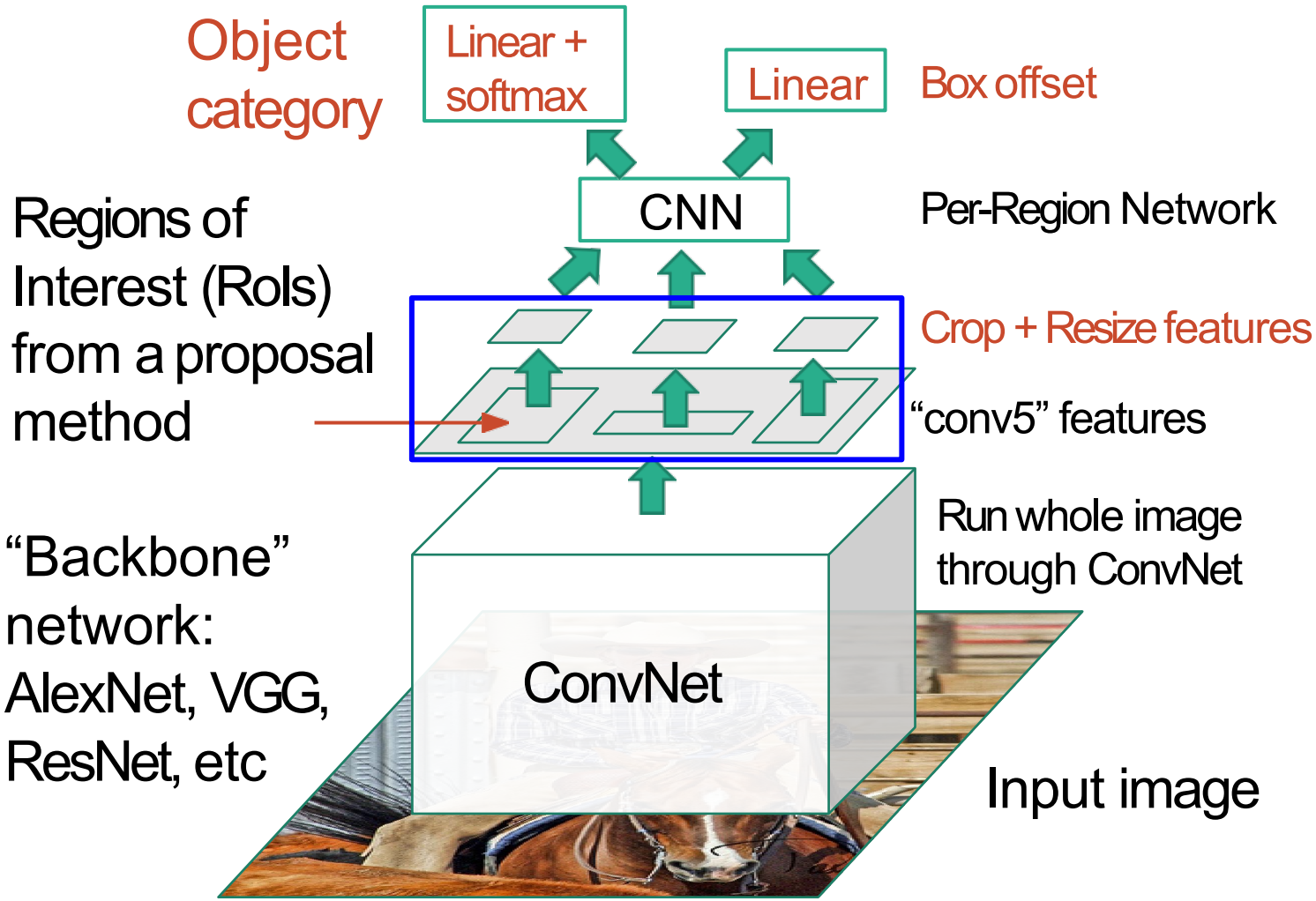
Problem: Very slow! Need to do ~2k independent forward passes for each image!

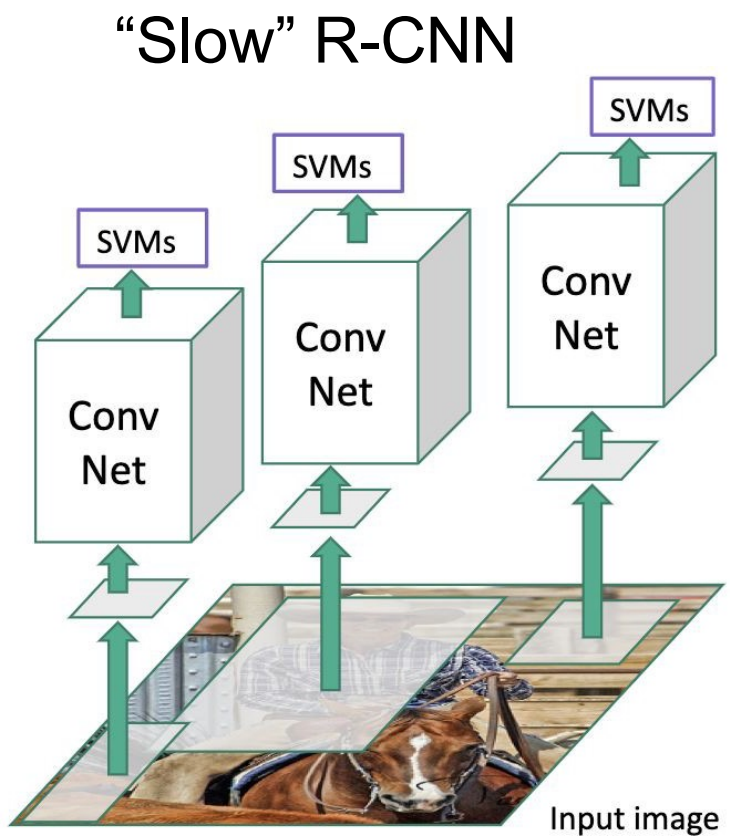
Idea: Pass the image through convnet before cropping! Crop the conv feature instead!

Girshick et al, "Rich feature hierarchies for accurate object detection and semantic segmentation", CVPR 2014.
Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Bbox reg: Predict "corrections" to the RoI: 4 numbers: (dx, dy, dw, dh)

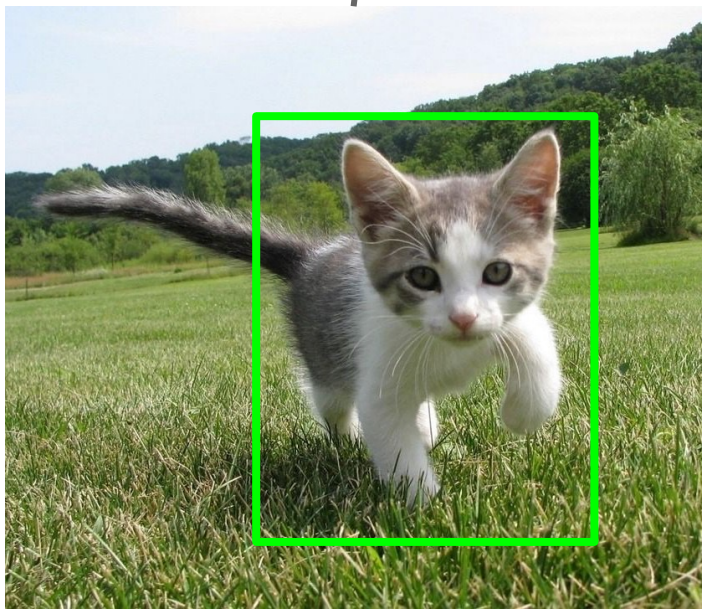
Fast R-CNN





Girshick, "Fast R-CNN", ICCV 2015. Figure copyright Ross Girshick, 2015; source. Reproduced with permission.

Cropping Features: RoI Pool



Input Image
(e.g. 3 x 640 x 480)

Project proposal onto features

CNN

"Snap" to grid cells

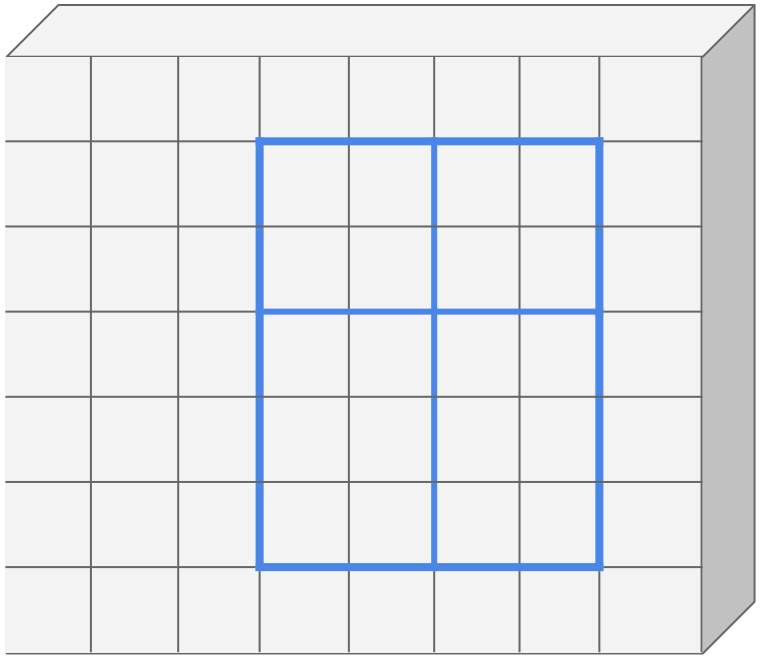
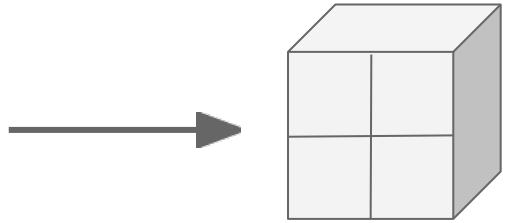


Image features: C x H x W
(e.g. 512 x 20 x 15)

Divide into 2x2 grid of (roughly) equal subregions

Max-pool within each subregion



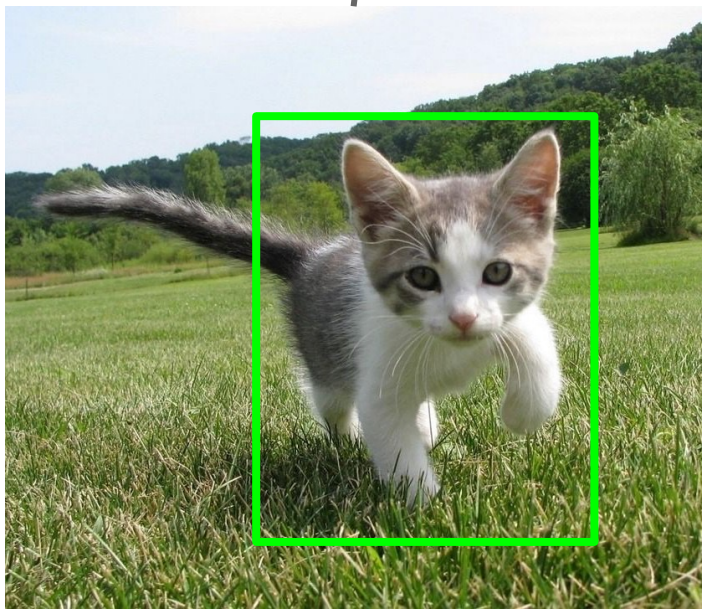
Region features
(here 512 x 2 x 2;
In practice e.g 512 x 7 x 7)

Region features always the same size even if input regions have different sizes!

Problem: Region features slightly misaligned

Girshick, "Fast R-CNN", ICCV 2015.

Cropping Features: RoI Align



Input Image
(e.g. 3 x 640 x 480)

Project proposal onto features

No “snapping”!

CNN

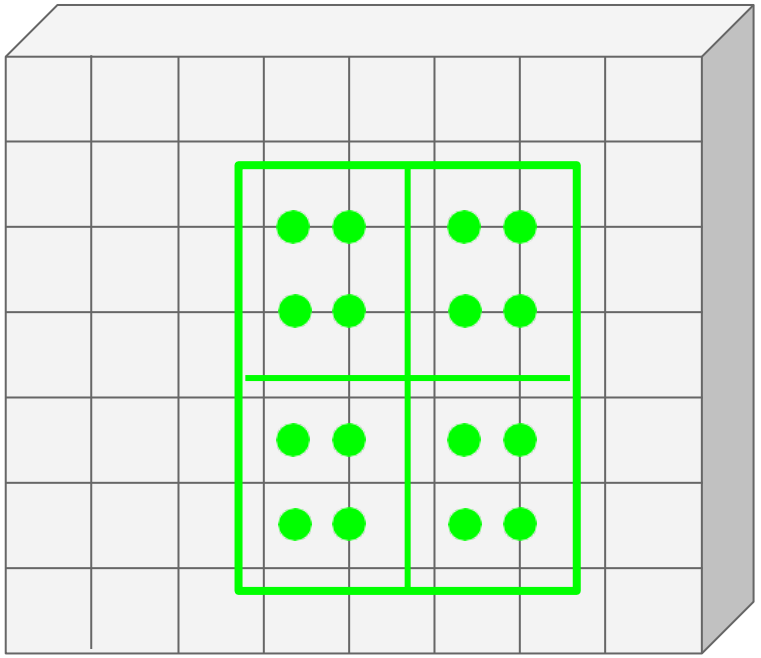
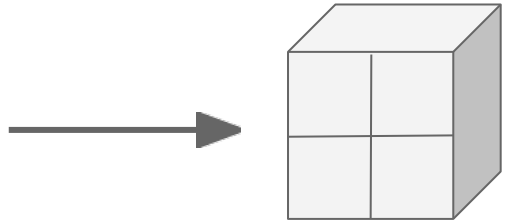


Image features: C x H x W
(e.g. 512 x 20 x 15)

Max-pool within each subregion

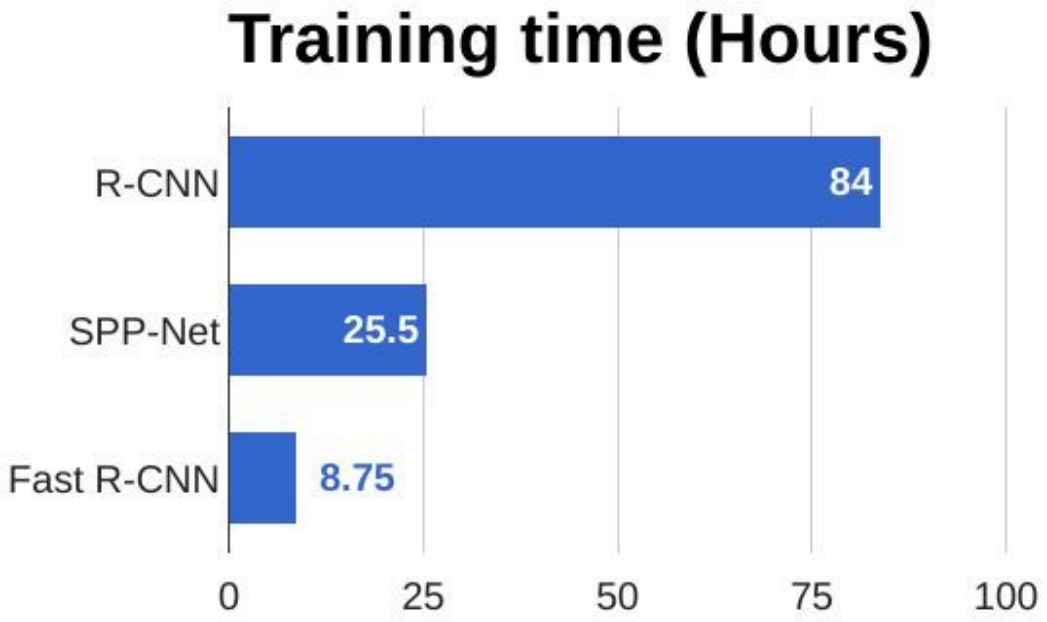


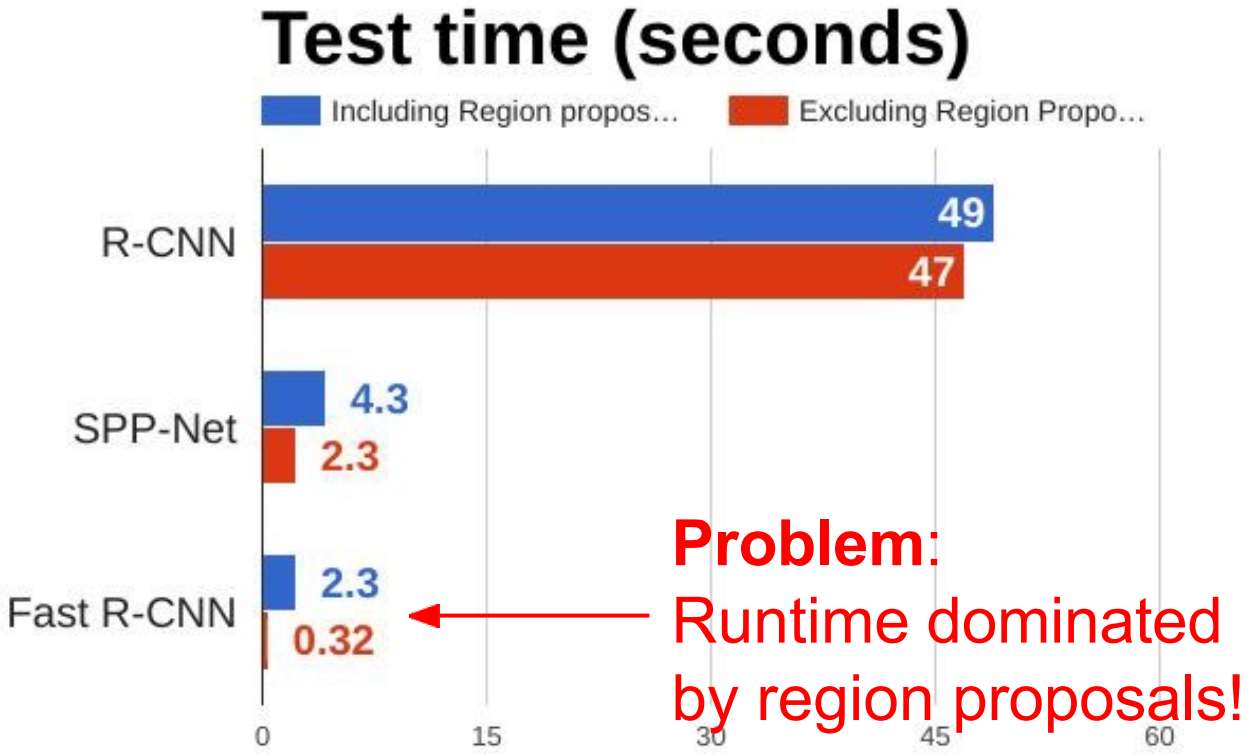
Region features
(here 512 x 2 x 2;
In practice e.g 512 x 7 x 7)

Sample at regular points in each subregion using bilinear interpolation

He et al, “Mask R-CNN”, ICCV 2017

R-CNN vs Fast R-CNN





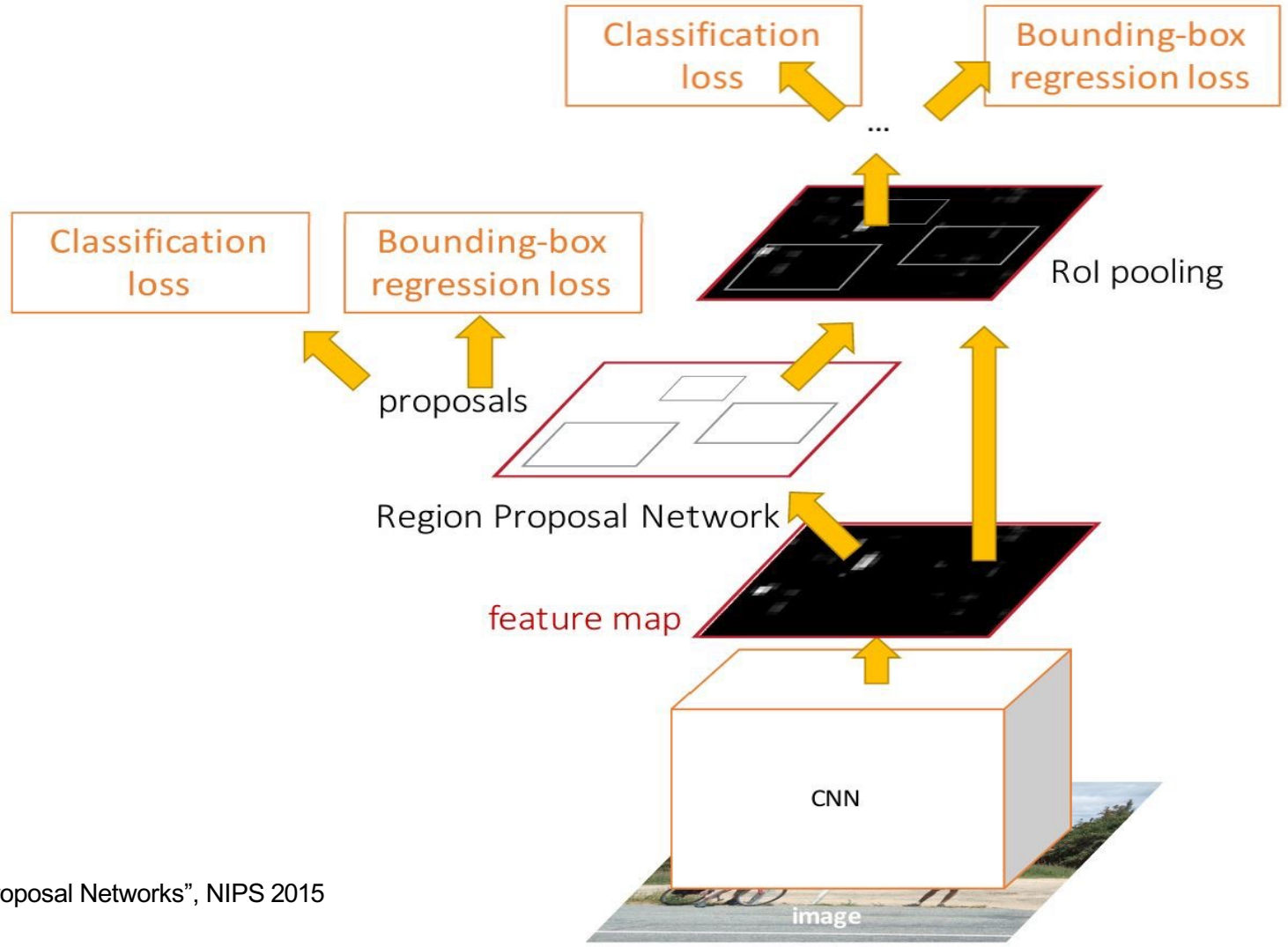
Problem:
Runtime dominated by region proposals!

Girshick et al, “Rich feature hierarchies for accurate object detection and semantic segmentation”, CVPR 2014.
He et al, “Spatial pyramid pooling in deep convolutional networks for visual recognition”, ECCV 2014
Girshick, “Fast R-CNN”, ICCV 2015

Faster R-CNN: Make CNN do proposals!

Insert **Region Proposal Network (RPN)** to predict proposals from features

Otherwise same as Fast R-CNN: Crop features for each proposal, classify each one



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

Region Proposal Network

In practice use K different anchor boxes of different size / scale at each point



Input Image
(e.g. 3 x 640 x 480)

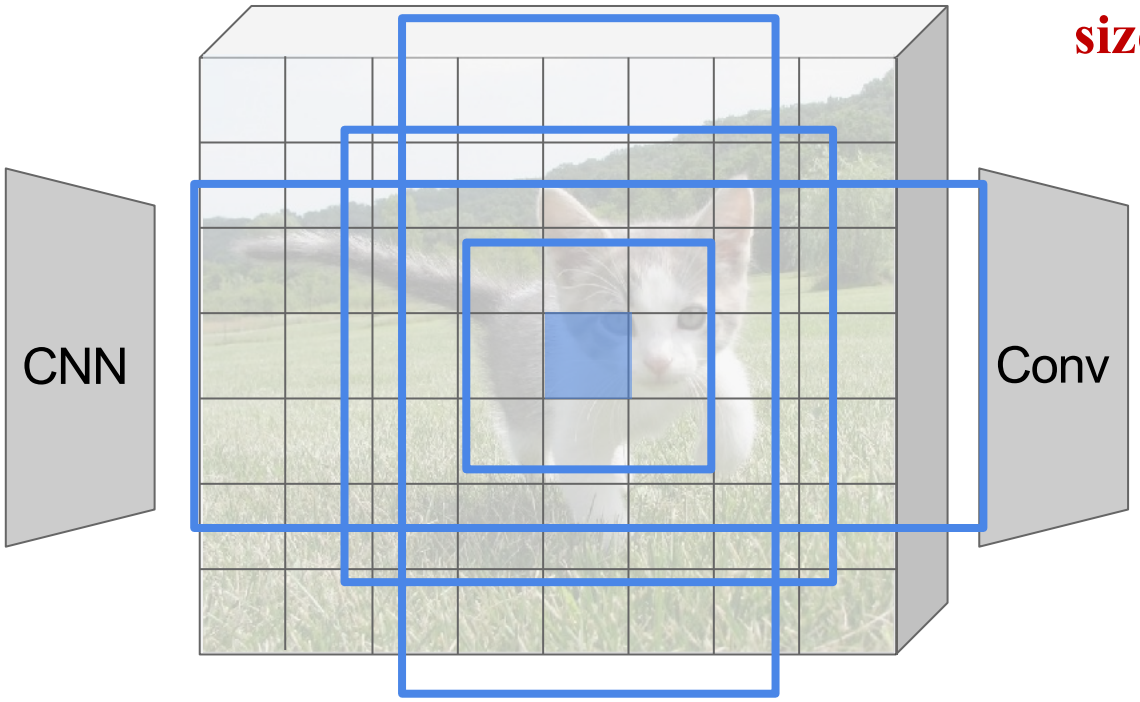


Image features
(e.g. 512 x 20 x 15)

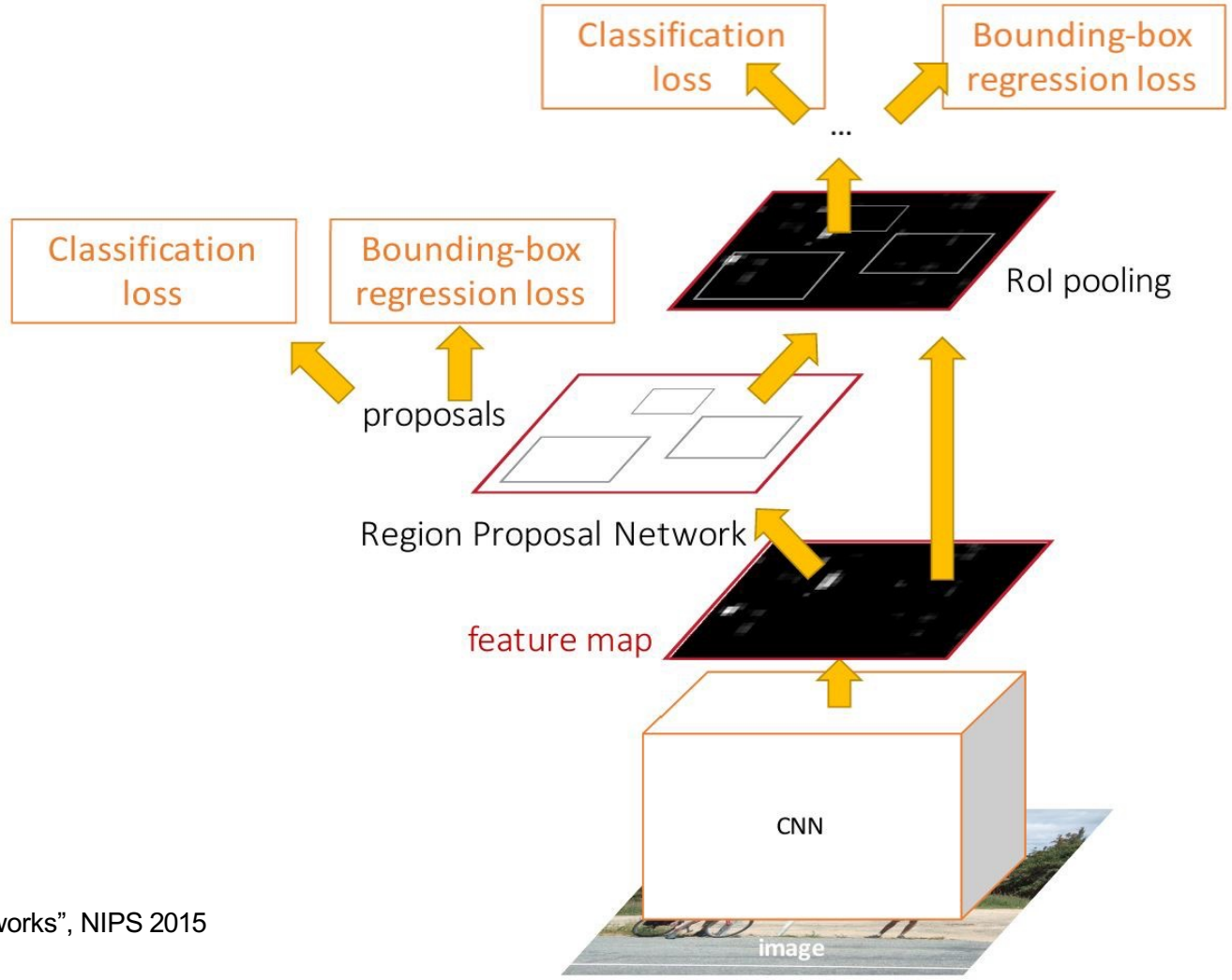
Anchor is an object?
K x 20 x 15

Box transforms
4K x 20 x 15

Faster R-CNN: Make CNN do proposals!

Jointly train with 4 losses:

1. RPN classify object / not object
2. RPN regress box coordinates
3. Final classification score (object classes)
4. Final box coordinates



Ren et al, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks", NIPS 2015
Figure copyright 2015, Ross Girshick; reproduced with permission

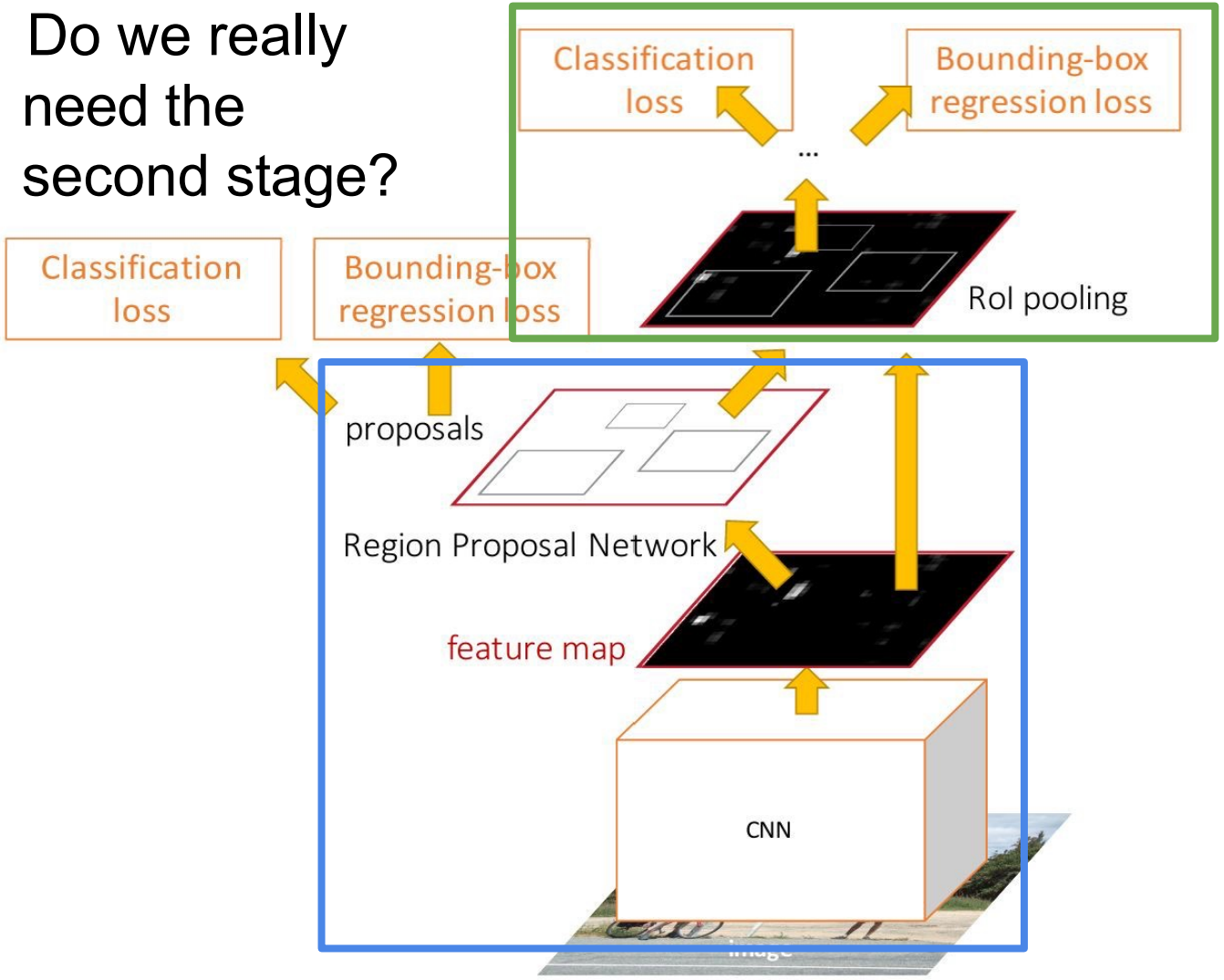
Faster R-CNN: Make CNN do proposals!

Faster R-CNN is a
Two-stage object detector

First stage: Run once per image
- Backbone network
- Region proposal network

Second stage: Run once per region
- Crop features: RoI pool / align
- Predict object class
- Prediction bbox offset

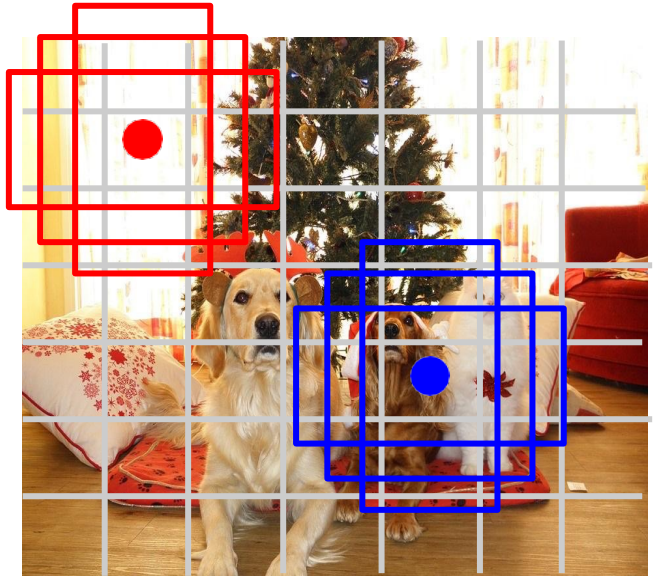
Do we really need the second stage?



Single-Stage Object Detectors: YOLO / SSD / RetinaNet



Input image
3 x H x W



Divide image into grid
7 x 7

Image a set of **base boxes** centered at each grid cell
Here B = 3

Within each grid cell:

- Regress from each of the B base boxes to a final box with 5 numbers: (dx, dy, dh, dw, confidence)
- Predict scores for each of C classes (including background as a class)
- Looks a lot like RPN, but category-specific!

Output:
7 x 7 x (5 * B + C)

Redmon et al, "You Only Look Once: Unified, Real-Time Object Detection", CVPR 2016
Liu et al, "SSD: Single-Shot MultiBox Detector", ECCV 2016
Lin et al, "Focal Loss for Dense Object Detection", ICCV 2017



YOLO: Detection as Regression

- **Grid:** S × S cells; the cell holding an object's center is responsible for it (Redmon et al., CVPR 2016).
- **Output:** per cell, objectness + box (center offset, log size) + C class scores: an S × S × (5 + C) tensor from one forward pass.
- **Loss:** objectness in every cell; box and class losses only in the responsible cells.
- **Inference:** score = objectness × class probability → threshold → NMS.
- **Later versions:** anchor boxes (v2), several output scales (v3), anchor-free heads (recent).



Object Detection: Lots of variables ...

Backbone Network
VGG16
ResNet-101
Inception V2
Inception V3
Inception ResNet
MobileNet

"Meta-Architecture"
Two-stage: Faster R-CNN
Single-stage: YOLO / SSD
Hybrid: R-FCN

Image Size
Region Proposals
…

Takeaways
Faster R-CNN is slower but more accurate

SSD is much faster but not as accurate

Bigger / Deeper backbones work better

Huang et al, "Speed/accuracy trade-offs for modern convolutional object detectors", CVPR 2017

R-FCN: Dai et al, "R-FCN: Object Detection via Region-based Fully Convolutional Networks", NIPS 2016
Inception-V2: Ioffe and Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift", ICML 2015
Inception V3: Szegedy et al, "Rethinking the Inception Architecture for Computer Vision", arXiv 2016
Inception ResNet: Szegedy et al, "Inception-V4, Inception-ResNet and the Impact of Residual Connections on Learning", arXiv 2016
MobileNet: Howard et al, "Efficient Convolutional Neural Networks for Mobile Vision Applications", arXiv 2017

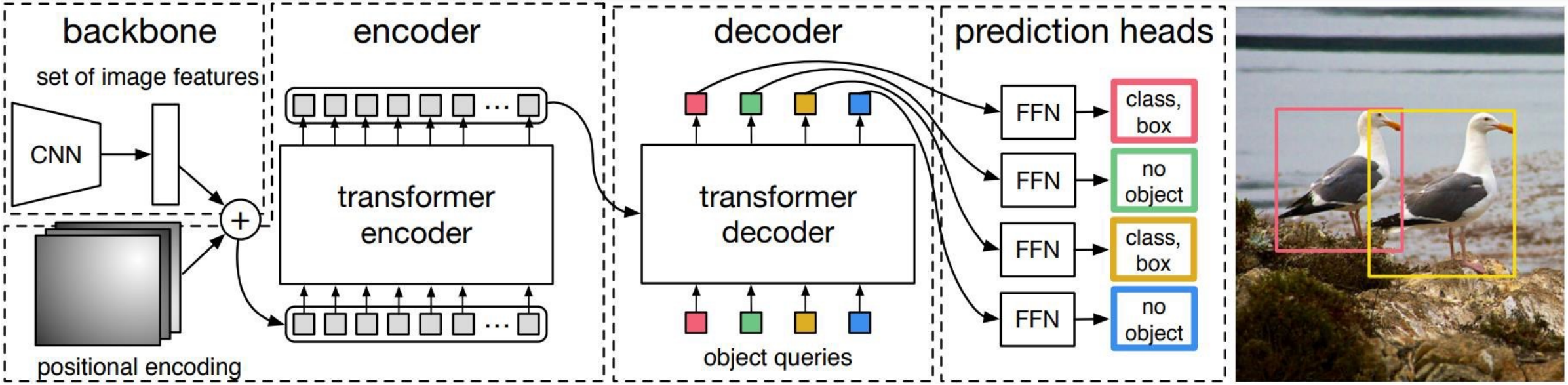


Notebook 28: Speed vs. Accuracy

- **SSDlite320 MobileNetV3** (one-stage): 3.4M params, COCO mAP 21.3, ~0.1 s per photo
- **Faster R-CNN MobileNetV3-FPN** (two-stage): 19.4M, mAP 32.8, ~0.3 s
- **RetinaNet ResNet-50-FPN v2** (one-stage): 38.2M, mAP 41.5, ~0.4 s
- **Faster R-CNN ResNet-50-FPN v2** (two-stage): 43.7M, mAP 46.7, ~1.4 s
- Tennis-team photo: SSDlite finds 7 people; the ResNet-50 Faster R-CNN finds 14, plus their bags.
- mAP: torchvision documentation. Times: one laptop CPU, 4 threads.

DETR - The First End-to-End Object Detector

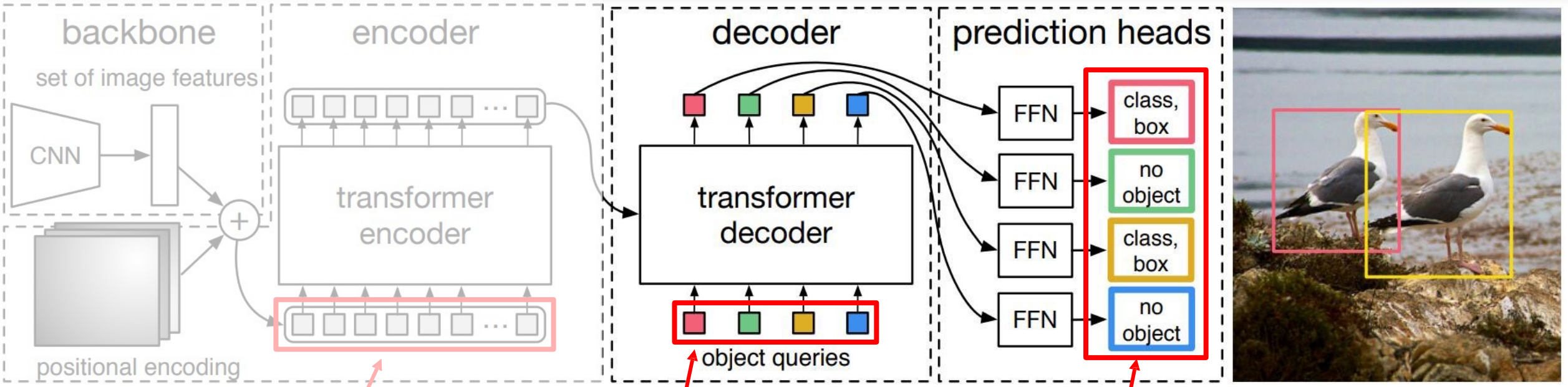
- Query-based Design



- Eliminate the need for hand-crafted components
- Achieve very competitive performance with previous modern object detectors

Carion, Nicolas, et al. "End-to-End Object Detection with Transformers." In ECCV 2020.

DETR - Architecture



$HW \times C$

flattened image features

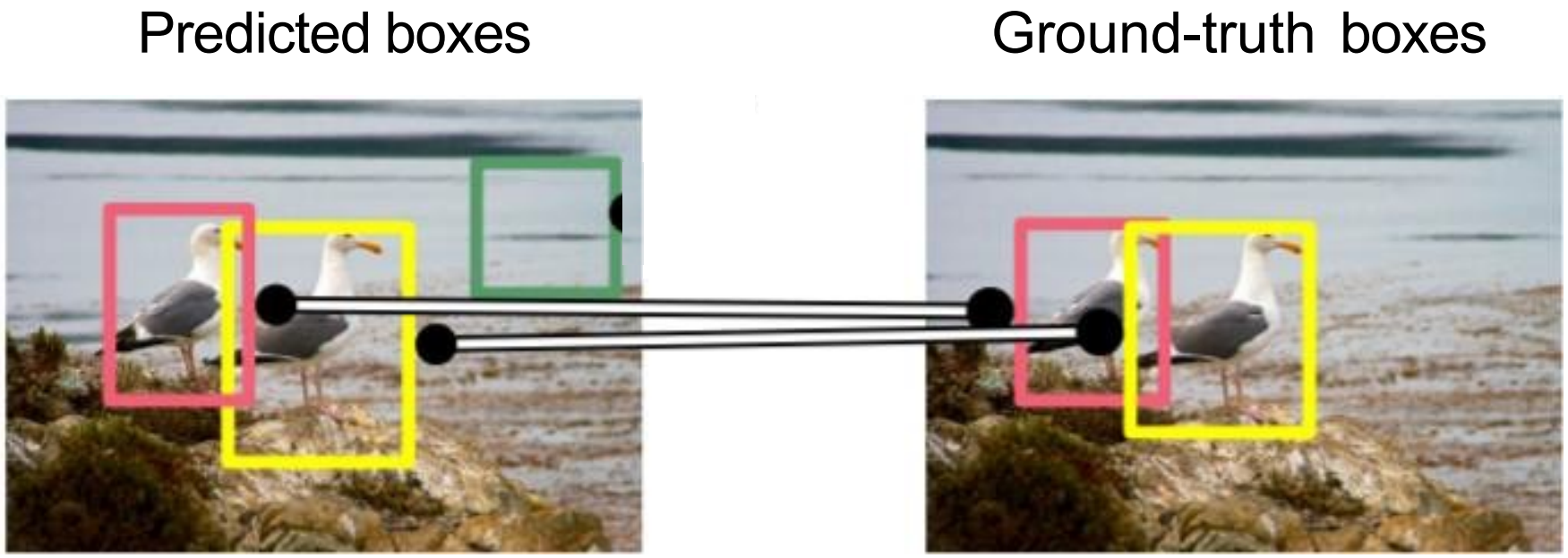
$N \times C$

learnable object queries (randomly initialized)

each object query predicts one bounding box

Carion, Nicolas, et al. "End-to-End Object Detection with Transformers." In ECCV 2020.

DETR - Training Target Assignment (Bipartite Matching)



Bipartite matching assignment to minimize the classification and regression costs

Carion, Nicolas, et al. "End-to-End Object Detection with Transformers." In ECCV 2020.



More

- https://github.com/open-mmlab/mmdetection



Summary and Next Lecture

- A detector outputs a variable-length list of (box, class, score). IoU defines a correct detection; NMS removes duplicates.
- Two-stage (Faster R-CNN): propose regions, then classify each. One-stage (YOLO, SSD, RetinaNet): predict from a grid in one pass, faster.
- DETR: object queries and bipartite matching; no anchors, no NMS.
- mAP: area under the precision–recall curve, averaged over classes (COCO: also over IoU 0.5–0.95).
- **Next (Dr. Wong, Mon 11/16):** segmentation, a label for every pixel.